

.NET Versions, Namespaces, .NET Core & Solutions

A Technical Report on the .NET Ecosystem

Topic	C# / .NET Fundamentals
Language	English

.NET Versions, Namespaces, .NET Core & Solutions

1. .NET Versions — How the Platform Evolved

".NET" is not one single product, it's an umbrella name that has covered several different implementations over the years. Understanding the timeline helps explain why terms like ".NET Framework" and ".NET Core" both exist, and why modern .NET (5 and above) unified them.

Era	Name	Key Characteristics
2002 – 2019	.NET Framework (1.0 → 4.8)	Windows-only, installed as part of the OS, tightly coupled to Windows APIs (WPF, WinForms, ASP.NET Web Forms).
2016 – 2020	.NET Core (1.0 → 3.1)	Cross-platform (Windows, Linux, macOS), open-source, modular, side-by-side installs, built for cloud and containers.
2020 – present	.NET (5, 6, 7, 8, 9, 10 ...)	The unification of .NET Framework and .NET Core into a single platform. ".NET Core" was dropped from the name starting with .NET 5.

Even Number was skipped: there was no ".NET 4" after Core 3.1, specifically to avoid confusion with the old ".NET Framework 4.x" line. From .NET 5 onward, Microsoft ships one release per year, with every even-numbered version (6, 8, 10 ...) marked as an **LTS (Long-Term Support)** release supported for 3 years, while odd-numbered versions (7, 9 ...) are **STS (Standard-Term Support)**, supported for 18 months.

Rule of thumb: for production systems, prefer an LTS version unless a specific feature in a newer STS release is required.

2. Namespaces

A namespace is a logical container used to organize related classes, interfaces, and types, and to avoid naming collisions. Instead of every class living in one giant global list, namespaces group them hierarchically, similar to folders organizing files.

- **Purpose:** avoid name clashes (e.g. two different libraries can each have a class called `Logger` as long as they sit in different namespaces).
- **Syntax:** declared with the `namespace` keyword, or in modern C# with a `file-scoped namespace` (a single line ending with `;` instead of braces).
- **Usage:** accessed with the `using` directive at the top of a file, so you don't need to fully qualify every type.
- **Convention:** namespaces typically mirror the folder structure of a project and often start with the company or product name, e.g. `Company.Product.Module`.

Example:

```
namespace MyCompany.Billing;

public class Invoice { }
```

3. .NET Core

.NET Core was Microsoft's ground-up rewrite of .NET, released in 2016, designed to fix the biggest limitations of the original .NET Framework.

Aspect	.NET Framework	.NET Core
Platform	Windows only	Windows, Linux, macOS
Source	Mostly closed-source (later partially open)	Fully open-source on GitHub
Deployment	Shared, machine-wide install	Can be self-contained per app, side-by-side versions
Performance	Good	Significantly faster (rewritten runtime and JIT improvements)
Use case today	Legacy enterprise apps	Superseded by modern .NET, ideal for new cloud/containerized apps

.NET Core's biggest impact was making .NET a serious option for cross-platform development and microservices, especially with Docker and Linux-based cloud infrastructure. This cross-platform, open-source foundation is what modern .NET (5+) is built on.

4. Solution (in Visual Studio / .NET tooling)

A **Solution** (`.sln` file) is a container that groups one or more related **Projects** together so they can be managed, built, and debugged as a single unit.

- A solution itself contains no code, it's essentially a manifest listing which projects belong together.
- A typical solution might include: a class library project, a Web API project, and a unit test project, all referencing each other.
- Solutions make it easy to work on multi-project systems (e.g. a backend API + a shared data-access library + a test suite) inside one IDE window.

- Modern .NET also supports `.slnx`, a newer, simpler XML-based solution file format.

— End of Report —